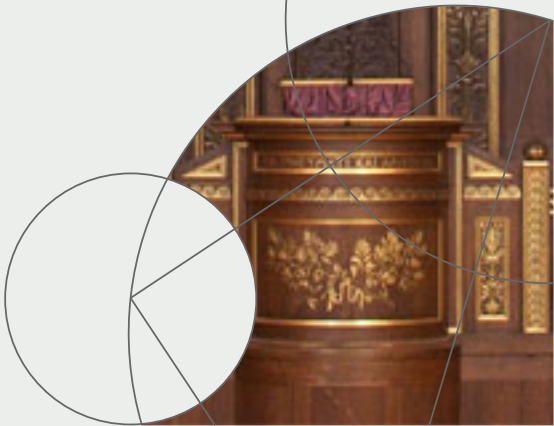
STEVENS INSTITUTE OF TECHNOLOGY

Data Acquisition and Processing I: Big Data

Rensheng Wang,
rwang1@stevens.edu
Dept. of Electrical and Computer Engineering
Stevens Institute of Technology



Why Feature Selection?

Question: Why do not we give all the features to the ML algorithm and let it decide which feature is important?

- Reason 1: Overfitting

  If we have more columns in the data than the number of rows, we will be able to fit our training data perfectly, but that won't generalize to the new samples.

  In other words, we learn both the model and the noises. If the noise features are dominant in our learning, then what we learned can be generalized to the new sets of data.

  It is very common to see the training set data achieves great performance while the testing data shows terrible results.

Why Feature Selection?

- Reason 2: Occam's Razor:

  We want our models to be simple and explainable. We lose explainability when we have a lot of features.

- Reason 3: Garbage In Garbage out:

  Most of the times, we will have many non-informative features. Poor-quality input will distract the machine learning process and produce Poor-Quality output

  In addition, it is obvious that a large number of features make the model training cost expensive, time-consuming and hard to implement.

☞ We select only useful features.

Feature Selection

Most feature selection methods can be divided into three major buckets

- **Filter based:**

  We specify some metric and based on that filter features. An example of such a metric could be correlation/chi-square.

- **Wrapper-based**

  Wrapper methods consider the selection of a set of features as a search problem. Example: Recursive Feature Elimination.

- **Embedded**

  Embedded methods use algorithms that have built-in feature selection methods. For instance, Lasso and RF have their own feature selection methods.

Pearson Correlation

- Pearson correlation is defined as

$$r = \frac{\sum(x-\bar{x})(y-\bar{y})}{\sqrt{\sum(x-\bar{x})^2 \sum(y-\bar{y})^2}}$$

  where $\bar{x}$ is the mean of sample x.

- This is a filter-based method.

  We check the absolute value of the Pearson's correlation between the target and numerical features in our dataset. We keep the top n features based on this criterion.

Pearson Correlation

- Sample code (Python)

```python
def cor_selector(X, y,num_feats):
    cor_list = []
    feature_name = X.columns.tolist()
    # calculate the correlation with y for each feature
    for i in X.columns.tolist():
        cor = np.corrcoef(X[i], y)[0, 1]
        cor_list.append(cor)
    # replace NaN with 0
    cor_list = [0 if np.isnan(i) else i for i in cor_list]
    # feature name
    cor_feature = X.iloc[:,np.argsort(np.abs(cor_list))[-num_feats:]].columns.tolist()
    # feature selection? 0 for not select, 1 for select
    cor_support = [True if i in cor_feature else False for i in feature_name]
    return cor_support, cor_feature
cor_support, cor_feature = cor_selector(X, y,num_feats)
print(str(len(cor_feature)), 'selected features')
```

Recursive Feature Elimination (RFE)

- This is a wrapper based method. As mentioned before, wrapper methods consider the selection of a set of features as a search problem.
- The goal of recursive feature elimination (RFE) is to select features by recursively considering smaller and smaller sets of features.
- First, the estimator is trained on the initial set of features and the importance of each feature is obtained either through a `coef_attribute` or through a `feature_importances_attribute`.
- Then, the least important features are pruned from current set of features. That procedure is recursively repeated on the pruned set until the desired number of features to select is eventually reached.

Recursive Feature Elimination (RFE)

- Herein we use `LogisticRegression` model, and the RFE observes the `coef_attribute` of the `LogisticRegression` object

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
rfe_selector = RFE(estimator=LogisticRegression(),
        n_features_to_select=num_feats, step=10, verbose=5)
rfe_selector.fit(X_norm, y)
rfe_support = rfe_selector.get_support()
rfe_feature = X.loc[:,rfe_support].columns.tolist()
print(str(len(rfe_feature)), 'selected features')
```

Lasso: SelectFromModel

- This is an Embedded method. As mentioned before, Embedded methods use algorithms that have built-in feature selection methods.
- For example, Lasso and RF(random forrest) have their own feature selection methods. Lasso Regularizer forces a lot of feature weights to be zero.
- Lasso regression use $L1$ norm as regularizer:

$$\lambda \sum_{i=1}^{k} |w_i|$$

  where w_i is the parameter.
- λ is the regularization parameter. Higher λ value, the fewer features have non-zero values.

Extra: Lasso Regression vs. Ridge Regression

- Let us start with a typical linear regression model: to predict the output y using a linear function of the input features x_i

$$\hat{y} = w_0 x_0 + w_1 x_1 + \cdots w_n x_n + b$$

- Linear regression looks for optimizing w_i and b such that it minimizes the cost function. The cost function can be written as

$$\sum_{i=1}^{M} (y_i - \hat{y}_i)^2 = \sum_{i=1}^{M} \left(y_i - \sum_{j=0}^{n} w_j \times x_{ij} - b \right)^2$$

- If we have very few features on a data-set and the score is poor for both training and test set then itâĂŹs a problem of under-fitting.
- On the other hand if we have large number of features and test score is relatively poor than the training score then it's the problem of over-generalization or over-fitting.

Extra: Lasso Regression vs. Ridge Regression

- Ridge and Lasso regression are some of the simple techniques to reduce model complexity and prevent over-fitting which may result from simple linear regression.
- The cost function for Lasso (least absolute shrinkage and selection operator) regression can be written as

$$\sum_{i=1}^{M}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{M}\left(y_i - \sum_{j=0}^{n} w_j \times x_{ij} - b\right)^2 + \lambda \sum_{j=0}^{n} |w_j|$$

- In ridge regression, the cost function is altered by adding a penalty equivalent to square of the magnitude of the coefficients.

$$\sum_{i=1}^{M}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{M}\left(y_i - \sum_{j=0}^{n} w_j \times x_{ij} - b\right)^2 + \lambda \sum_{j=0}^{n} w_j^2$$

Lasso: SelectFromModel

- Sample code (Python):

```
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LogisticRegression

embeded_lr_selector = SelectFromModel(LogisticRegression(penalty="l1"),
        max_features=num_feats)
embeded_lr_selector.fit(X_norm, y)

embeded_lr_support = embeded_lr_selector.get_support()
embeded_lr_feature = X.loc[:,embeded_lr_support].columns.tolist()
print(str(len(embeded_lr_feature)), 'selected features')
```

Tree-based: SelectFromModel

- This is an Embedded method. As mentioned before, Embedded methods use algorithms that have built-in feature selection methods.
- We can also use **RandomForest** to select features based on feature importance.
- The feature importances are essentially the mean of the individual trees' improvement in the splitting criterion produced by each variable. In other words, it is how much the score (so-called "impurity" on the decision tree notation) was improved when splitting the tree using that specific variable.
- They can be used to rank features and then select a subset of them.

Tree-based: SelectFromModel

- Sample code (Python)

```
from sklearn.feature_selection import SelectFromModel
from sklearn.ensemble import RandomForestClassifier

embeded_rf_selector =
SelectFromModel(RandomForestClassifier(n_estimators=100),
        max_features=num_feats)
embeded_rf_selector.fit(X, y)

embeded_rf_support = embeded_rf_selector.get_support()
embeded_rf_feature = X.loc[:,embeded_rf_support].columns.tolist()
print(str(len(embeded_rf_feature)), 'selected features')
```